

WEEK 4

Advanced Network Security Practical Assessment Report

Technical Learning & Practical Documentation

Assessment focus

Authorized lab network setup, web-service exposure, Nmap discovery, packet-level Wireshark analysis, troubleshooting, and evidence-based security interpretation.

Intern	SAMYUKTHA S.
Internship ID	DG/SEPTEMBER/CYBER/102
Organization	DG Interns Hub
Platform	VMware Workstation
Lab systems	Kali Linux + Ubuntu Server
Assessment date	September 2026

Authorized and isolated practical environment

1. Introduction & Assessment Approach

Why the lab was built and how the assessment was performed

This Week 4 exercise was designed as a small, controlled network-security assessment. Instead of treating each tool as an isolated requirement, the work was organized as a single workflow: establish a known network, introduce a service, observe how its exposure changes, inspect the resulting packets, troubleshoot unexpected behavior, and validate the final state.

Objectives

- Build and verify a private VMnet1 lab network between Kali Linux and Ubuntu Server.
- Configure Apache on the Ubuntu target and verify that HTTP is actually reachable from Kali.
- Use Nmap to identify exposed ports and service/version information, including a controlled before/after experiment.
- Use Wireshark to inspect ICMP, DNS, and HTTP traffic at packet level.
- Correlate configuration state, Nmap observations, and packet evidence without treating every exposure as a confirmed vulnerability.

Assessment method

Stage	Question	Evidence
1 - Network	Can the two VMs communicate on the intended lab network?	IP/routing + bidirectional ping
2 - Service	What changes when Apache is introduced?	Apache state, listener, UFW, HTTP access
3 - Discovery	Can the new service be discovered remotely?	Nmap baseline, after-scan, -sV
4 - Traffic	What happens on the wire?	ICMP, DNS, HTTP captures
5 - Correlation	Do local service state and remote observations agree?	Final Nmap + service + HTTP validation

Analyst rule

The report distinguishes observation, exposure, potential security significance, and confirmed vulnerability. An open port by itself is not treated as proof of a vulnerability.

2. Lab Architecture & Advanced Topology

Exact final network configuration used during the practical

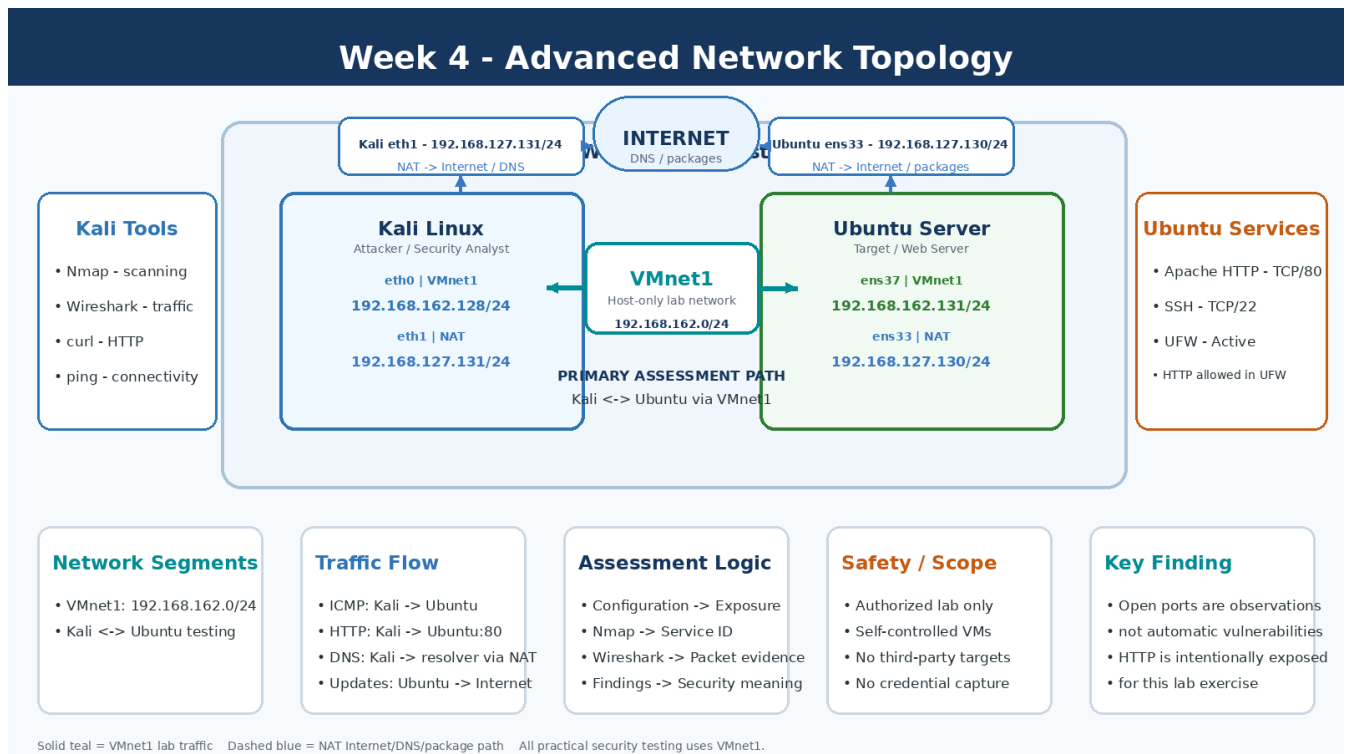


Figure 1. Final Week 4 advanced topology - VMnet1 carries Kali ↔ Ubuntu assessment traffic; NAT provides separate Internet/DNS/package access.

VMnet1 / Host-only	192.168.162.0/24 - primary authorized testing network
Kali eth0	192.168.162.128/24 - VMnet1
Kali eth1	192.168.127.131/24 - NAT
Ubuntu ens37	192.168.162.131/24 - VMnet1
Ubuntu ens33	192.168.127.130/24 - NAT
Target services	SSH TCP/22; Apache HTTP TCP/80
Security control	UFW active; TCP/22 and TCP/80 allowed

Why two paths?

VMnet1 keeps the security-testing path explicit and isolated. NAT was used separately for Internet, DNS, and package access so the target could install software and the Kali host could perform the DNS experiment without changing the lab-testing path.

Configuration evidence

Figure 2. Kali IP/routing state

Figure 3. Ubuntu IP/routing state

Figure 4. Kali → Ubuntu: 0% packet loss

Figure 5. Ubuntu → Kali: 0% packet loss

Analyst significance

Week 4 - Network Security Practical Task | Page 4

Apache was installed through the Ubuntu NAT path after package availability and Internet/DNS connectivity were verified. The service was then validated locally, at the socket layer, through the firewall, and from Kali.

Figure 6. Apache listening on TCP/80

Figure 7. UFW allows TCP/80

Figure 8. Kali retrieves Apache HTML from 192.168.162.131 over VMnet1.

Troubleshooting lesson

Week 4 - Network Security Practical Task | Page 5

5. Task 3 - Nmap Baseline & Controlled Before/After

Measuring the effect of introducing Apache

The controlled comparison kept the target at 192.168.162.131 and changed only Apache service state. This makes the result more meaningful than comparing different hosts or addresses.

```
kali@kali:~$ nmap -T4 192.168.162.131
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-23 03:30 -0400
Nmap scan report for 192.168.162.131
Host is up (0.00094s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    closed http
MAC Address: 00:0C:29:76:16:1D (VMware)

Nmap done: 1 IP address (1 host up) scanned in 9.32 seconds
kali@kali:~$
```

Figure 9. Apache stopped - 80/tcp closed

```
kali@kali:~$ nmap -T4 192.168.162.131
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-23 03:35 -0400
Nmap scan report for 192.168.162.131
Host is up (0.00095s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 00:0C:29:76:16:1D (VMware)

Nmap done: 1 IP address (1 host up) scanned in 9.50 seconds
kali@kali:~$
```

Figure 10. Apache started - 80/tcp open

Condition	22/tcp	80/tcp	Interpretation
Apache stopped	open / ssh	closed / http	Firewall permits 80, but no HTTP process is listening
Apache started	open / ssh	open / http	Apache is listening and reachable on TCP/80

The change in port state is directly explained by the service-state change: when Apache is stopped, no application accepts TCP/80; once Apache starts, the same port becomes observable as open. This is a concrete demonstration of service exposure.

Security significance

Service exposure is created by the interaction of application state and network policy. A firewall rule alone does not make a service exist, and an application listener alone does not guarantee remote reachability.

6. Nmap Service & Version Detection

Moving from port state to software identification

```
kali@kali:~ % nmap -T4 -sV -p 22,80 192.168.162.131
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-23 03:37 -0400
Nmap scan report for 192.168.162.131
Host is up (0.00064s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.2p1 Ubuntu 2ubuntu3.5 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.66 ((Ubuntu))
MAC Address: 00:0C:29:76:16:1D (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 10.91 seconds

kali@kali:~ %
```

Figure 11. Nmap -sV identifies SSH and Apache on the target.

Port	State	Service	Observed version
22/tcp	open	ssh	OpenSSH 10.2p1 Ubuntu 2ubuntu3.5
80/tcp	open	http	Apache httpd 2.4.66 (Ubuntu)

Service/version detection provides more context than a simple port scan. In this lab, Nmap identified both the service type and the responding software. The version strings are observations; they do not, by themselves, establish exploitability or a confirmed vulnerability.

```
kali@kali:~ % nmap -T4 192.168.162.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-23 02:05 -0400
Nmap scan report for 192.168.162.130
Host is up (0.00073s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 00:0C:29:76:16:13 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 9.40 seconds

kali@kali:~ %
```

Figure 12. Earlier baseline before the final target IP was established; retained as original lab evidence.

Evidence handling

The earlier 192.168.162.130 scan is retained for provenance. The 192.168.162.131 stopped/started comparison is the controlled before/after experiment used for analysis.

7. Task 4 - Wireshark: ICMP Packet Analysis

Reading reachability traffic at frame, IP, and ICMP levels

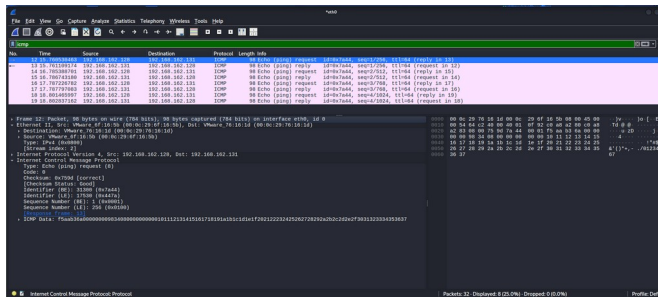


Figure 13. ICMP Echo Request

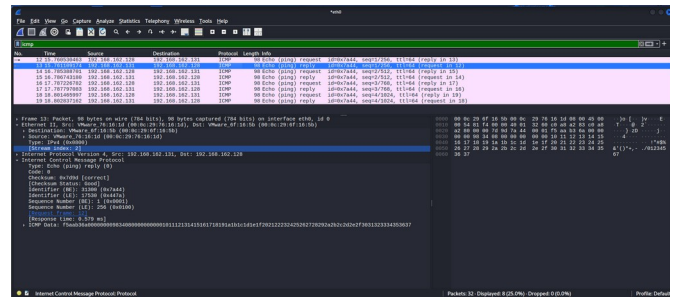


Figure 14. ICMP Echo Reply

What the packets show

Layer	Observed evidence	Meaning
Ethernet II	Source/Destination VMware MACs	Local virtual Ethernet delivery
IPv4	192.168.162.128 → 192.168.162.131	Kali sends the reachability test to Ubuntu
ICMP request	Type 8, Code 0	Echo Request
ICMP reply	Type 0, Code 0	Echo Reply confirms response
Correlation	Matching identifier/sequence	Reply can be tied to the originating request

The capture demonstrates the packet path behind the ping command. Instead of treating ping as a single “success” message, the analyst can see the Ethernet, IPv4, and ICMP layers and correlate the request with the reply.

Security significance

ICMP is useful for reachability and path validation. In an investigation, repeated or unusual ICMP patterns can be worth examining, but a normal echo request/reply pair alone is not evidence of malicious activity.

8. Task 4 - Wireshark: DNS Analysis & Troubleshooting

Using packet evidence to distinguish a query from a working DNS exchange

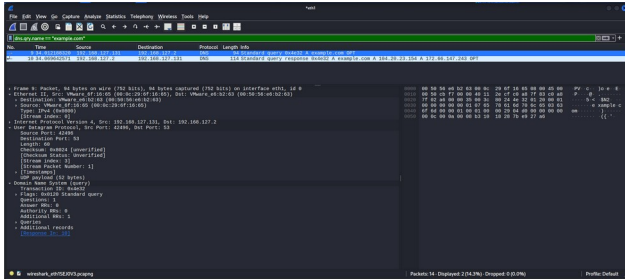


Figure 15. DNS query for example.com

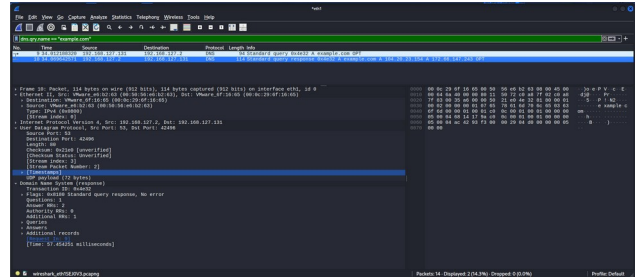


Figure 16. DNS response for example.com

Field	Observed result	Interpretation
Transport	UDP	DNS request/response used UDP
Port	53	Standard DNS service port
Name	example.com	Controlled lookup
Resolver	192.168.127.2 via NAT	Working resolver
Response	NOERROR / A records	Successful resolution

Troubleshooting path

- DNS queries were initially visible on eth0, but 192.168.162.1 did not provide a usable response.
- Kali resolv.conf and routing showed 192.168.162.1 as the resolver on the lab interface.
- A temporary NAT interface provided Internet/DNS access through 192.168.127.2 while VMnet1 remained the testing path.
- A fresh capture on the NAT interface produced the intended query/response pair.

Analyst lesson

A DNS query on the wire is not proof that resolution works. Resolver availability, routing, and the returned response must agree.

9. Task 4 - Wireshark: HTTP & TCP/Application Correlation

From TCP connection establishment to GET request and 200 OK response

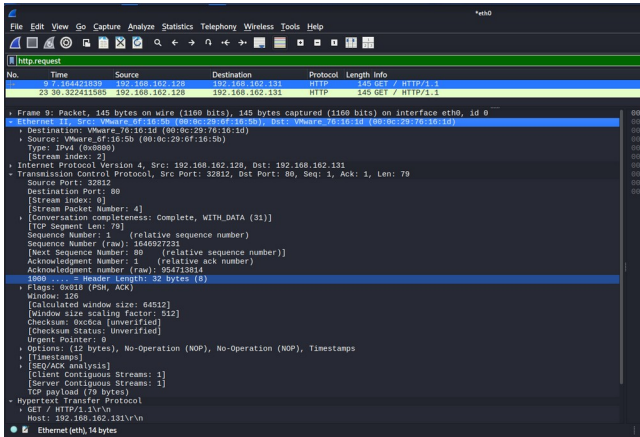


Figure 17. HTTP GET request

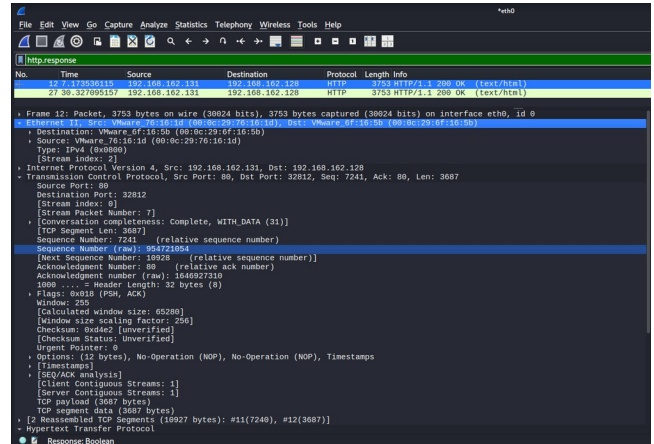


Figure 18. HTTP 200 OK response

Layer	Request-side evidence	Response-side evidence
IPv4	192.168.162.128 → 192.168.162.131	192.168.162.131 → 192.168.162.128
TCP	32812 → 80	80 → 32812
Application	GET / HTTP/1.1; Host: 192.168.162.131	HTTP/1.1 200 OK; text/html
Payload	79-byte TCP segment	3687-byte segment; reassembly noted

The HTTP capture is the clearest cross-tool correlation in the project. Nmap established that TCP/80 was open and identified Apache. Wireshark then showed the actual client/server exchange that produced the successful web response. The application data sits above TCP, so the packet view can be read as Ethernet → IPv4 → TCP → HTTP.

Security significance

Plain HTTP exposes application-layer request/response content to any party that can capture the traffic path. In this controlled lab the traffic is intentional and authorized; in production, encrypted HTTPS is normally used for sensitive application data.

```
targetaddnshuntu-target%$ systemctl is-active apache2
active
targetaddnshuntu-target%$ sudo df -sh
Status: active

To Action From
-----
22/tcp ALLOW Anywhere
22/tcp (s0) ALLOW Anywhere (s0)
22/tcp (s0) ALLOW Anywhere (s0)
22/tcp (s0) ALLOW Anywhere (s0)

targetaddnshuntu-target%$ sudo ss -ttnp | grep -E '[:22]08'
LISTEN 0 4096 0.0.0.0:22 0.0.0.0:* users:(("sshd",pid=1461,f0d3),("systemd",pid=1,fd=245))
LISTEN 0 512 0.0.0.0:22 0.0.0.0:* users:(("sshd",pid=1461,f0d4),("sshd",pid=3114,f0d4),("sshd",pid=3116,f0d4))
LISTEN 0 4096 0.0.0.0:22 0.0.0.0:* users:(("sshd",pid=1461,f0d4),("systemd",pid=1,fd=246))

targetaddnshuntu-target%$ _
```

```
kali@kali:~$ nmap -T4 -sV -p 22,80 192.168.162.131
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-23 07:49 -0400
Nmap scan report for 192.168.162.131
Host is up (0.0016s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 2ubuntu3.5 (Ubuntu Linux; protocol 2.0)
80/tcp    open  httpd    Apache/2.4.66 ((Ubuntu))
MAC Address: 00:0C:29:76:16:1D (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 8.83 seconds

kali@kali:~$
```

```
kali@kali: ~  
Session Actions Edit View Help  
  
kali@kali:~$ curl --connect-timeout 10 http://192.168.162.131  
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/  
DTD/xhtml1-transitional.dtd">  
<html xmlns="http://www.w3.org/1999/xhtml">  
  <!-- Modified from the Debian original for Ubuntu -->  
    Last updated: 2022-03-22  
    See: https://launchpad.net/bugs/1966004  
--> [Status: 200 OK] [Size: 1036] [Time: 0.000] [Interface: eth0, id 0]  
<head>  
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />  
  <title>Apache2 Ubuntu Default Page: It works</title>  
  <style type="text/css" media="screen"> Body {background-color: #d8dbed; border: 1px solid #ccc; margin: 0px 0px 0px 0px; padding: 0px 0px 0px 0px;}  
* {margin: 0px 0px 0px 0px; padding: 0px 0px 0px 0px;}  
[Conversation completeness: complete, WITH_DATA (31)]  
TCP segment Len=3687  
body, html {border: 1px solid #ccc; position: relative; width: 100%; height: 100%; padding: 3px 3px 3px 3px; background-color: #d8dbe2; font-family: Ubuntu, Verdana, sans-serif; font-size: 11pt; text-align: center;}  
[Window size scaling factor=250]  
Checksum: b9d4e2 [unverified]  
Content Status: Unverified  
div.main_page {position: relative; display: table; width: 800px; margin-bottom: 3px; margin-left: auto; margin-right: auto;}  
[Response Details]
```

Category	Evidence-based finding	Support
Observation	SSH listening on TCP/22	ss + Nmap -sV
Observation	Apache listening on TCP/80	ss + Nmap -sV
Exposure	Services reachable from Kali on VMnet1	Final Nmap
Potential risk	Reachable services add to attack surface; HTTP is unencrypted	Nmap + Wireshark
Confirmed vulnerability	None established by this exercise	No exploit/vulnerability proof performed

The mini project links target configuration to what an external scanner sees and what the network packets reveal. This closes the loop from setup to evidence.

11. Learning, Limitations & Conclusion

What the practical demonstrated beyond the checklist

What I learned

- Virtual network design is a security control: VMnet1 provided a deliberate Kali ↔ Ubuntu testing path.
- A port state is not the same as a running service; the Apache stopped/started experiment demonstrated this directly.
- Nmap summarizes exposed services, while Wireshark shows packet behavior that supports the observation.
- Troubleshooting should move from configuration to routing and service evidence rather than changing settings blindly.
- Security findings should be stated at the level supported by evidence; an open port is not automatically a vulnerability.

Limitations

- The assessment was limited to the two self-controlled VMs and services intentionally configured for the assignment.
- No exploitation, credential capture, or third-party scanning was performed.
- Version identification was treated as reconnaissance evidence, not proof of exploitability.

Conclusion

Week 4 demonstrated a complete basic network-security assessment workflow in an authorized VM lab: network establishment, service configuration, discovery, packet analysis, troubleshooting, and final validation. The strongest evidence was the controlled Apache experiment, where TCP/80 changed from closed to open after the service started, followed by service identification and Wireshark confirmation of the resulting HTTP exchange.

Final takeaway

The work moved from “running commands” to “explaining evidence.” That is the practical skill carried forward into SOC and blue-team learning.

End of Week 4 Technical Report